Expert Systems with Applications 36 (2009) 3824–3828

Contents lists available at ScienceDirect

Expert Systems with Applications

journal homepage: www.elsevier.com/locate/eswa

Constructing investment strategy portfolios by combination genetic algorithms

Jiah-Shing Chen [a], Jia-Li Hou [b,*], Shih-Min Wu [a], Ya-Wen Chang-Chien [a]

[a] Department of Information Management, National Central University, No. 300, Jhongda Road, Jhongli City, Taoyuan County 32001, Taiwan, ROC
[b] Department of Information Management, National Dong Hwa University, No. 1, Section 2, Da Hseuh Road, Shoufeng, Hualien, Taiwan, ROC

ARTICLE INFO

Keywords:
Genetic algorithms (GA)
Portfolio
Capital allocation
Combination genetic algorithm
Investment strategy portfolio

ABSTRACT

The classical portfolio problem is a problem of distributing capital to a set of securities. By generalizing the set of securities to a set of investment strategies (or security-rule pairs), this study proposes an investment strategy portfolio problem, which becomes a problem of distributing capital to a set of investment strategies. Since the investment strategy portfolio problem can be formulated as a combination optimization problem, a new combination genetic algorithm is proposed for solving the new investment strategy portfolio problem. Experimental results show that the idea of investment strategy portfolios is feasible and the combination genetic algorithm is effective on the investment strategy portfolio problem.

1. Introduction

The classical portfolio problem is a problem of distributing capital to a set of securities (Gondzio & Grothey, 2007; Ince & Trafalis, 2006; Markowitz & Arnott, 1952; Wu & Chang, 2007). By generalizing the set of securities to a set of investment strategies (or security-rule pairs), this study proposes an investment strategy portfolio problem, which becomes a problem of distributing capital to a set of investment strategies. The classical portfolio problem can be viewed a special case of the new investment strategy portfolio problem with buy-and-hold as the only trading rule.

Both the investment strategy portfolio problem and the classical portfolio problem can be formulated as combination optimization problems. Therefore, a new combination genetic algorithm (CGA) is proposed for solving the combination optimization problem in general, and the new investment strategy portfolio problem in particular.

Statistical test result indicates that the performance of our CGA is significantly better than that of uniform allocation. Experimental results show that the idea of investment strategy portfolios is feasible and the combination genetic algorithm is effective on the investment strategy portfolio problem.

The rest of this paper is organized as follows. Section 2 reviews the classical portfolio problem and genetic algorithms. Section 3 describes the investment strategy portfolio problem and our solution method, the combination genetic algorithm. Section 4 presents the results of our experiments. Section 5 gives the conclusions and future directions.

2. Background

2.1. Classical portfolio optimization problem

The classical portfolio optimization problem can be formulated as follows:

$$\begin{aligned} &\max && \alpha \sum_i x_i r_i - (1-\alpha) \sum_{ij} x_i x_j \sigma_{ij} \\ &\text{subject to} && \sum_{i=0}^{n} x_i = 1, 0 \leqslant x_i \leqslant 1, x_i \in R, \end{aligned} \tag{1}$$

where α is a real constant between 0 & 1, x_i is the proportion (between 0 and 1) of capital invested in instrument i, r_i is the expected return rate of instrument i and σ_{ij}^2 is the covariance of r_i and r_j. The objective is to find the optimal portfolio with maximum return and/or minimum risk.

Traditional approach to this portfolio problem is Markowitz's mean–variance analysis which uses the Lagrange multiplier method to find the optimal static portfolio (Kim & Markowitz, 1989; Markowitz & Arnott, 1952).

2.2. Genetic algorithms on portfolio optimization problem

Genetic algorithms (GA) were proposed by Holland in 1975 from Darwin's theory of evolution, i.e., survival of the fittest (Darwin, 1859; Holland, 1975). Genetic algorithms uses an evolutionary process resulting in a fittest solution to solve a problem. The evolutionary process consists of several genetic operators:

* Corresponding author. Tel.: +886 932 228466; fax: +886 3 863100.
E-mail addresses: jschen@ncu.edu.tw (J.-S. Chen), alexhou@mail.ndhu.edu.tw (J.-L. Hou), 93423012@cc.ncu.edu.tw (S.-M. Wu), 92443001@cc.ncu.edu.tw (Y.-W. Chang-Chien).

0957-4174/$ - see front matter © 2008 Elsevier Ltd All rights reserved.
doi:10.1016/j.eswa.2008.02.019

selection, crossover and mutation (Bäck, Fogel, & Michalewicz, 2000a, 2000b; De Jong, 2002; Goldberg, 1989; Mitchell, 1996; Srinivas & Patnaik, 1994).

Genetic algorithms are computationally simple and powerful. Genetic algorithms are very good tool for optimization problems since they make no restrictive assumptions about the solution space.

To solve a problem with genetic algorithms, an encoding mechanism must first be designed to represent each solution as a chromosome, e.g., a binary string. A fitness function is also required to measure the goodness of a chromosome. Genetic algorithms search the solution space using a population which is simply a set of chromosomes. During each generation, the three genetic operators: selection, crossover and mutation, are applied to the population several times to form a new population. Selection picks 2 chromosomes according to their fitness: a fitter chromosome has a higher probability of being selected. Crossover recombines the 2 selected chromosomes to form new offspring with a crossover rate. Mutation randomly alters each position in each offspring with a small mutation rate. New population is then generated by replacing some chromosomes with new offspring. This process is repeated until some termination condition, e.g., the number of generations, is reached. Fig. 1 shows the pseudo code of the basic genetic algorithm. When the number of genetic applications (k) is half the population size ($n/2$), the GA is called generational GA; when $k < n/2$, it is called steady-state GA.

The advantage of GAs is in their parallelism. GA searches a solution space using a population of individuals so that they are less likely to get stuck in local optimums. This is achieved with a cost, i.e., the computational time. GAs can be slower than other methods. However, the longer run time of GAs can be shortened by terminating the evolution earlier to get a satisfactory solution.

Genetic algorithms have been applied to various domains over the years (Beasley, 2000; Oh, Kim, & Min, 2005; Oh, Kim, Min, & Lee, 2006). Financial applications of genetic algorithms are starting to show promising results. Bauer used genetic algorithms to generate trading rules which are Boolean expressions with 3 of the 10 allowed time series (Bauer, 1994). Colin applied genetic algorithms to find the lengths in the moving average crossover strategy (Colin, 1994). Deboeck studied methods of using genetic algorithms to train a neural network trading system (Deboeck, 1994).

The easiest way to use GAs on the portfolio problem is to encode each weight x_i as a non-negative integer or floating number. Standard genetic operators can then be applied as usual. To enforce the $\sum_{i=0}^{n} x_i = 1$ constraint, normalization of each x_i by dividing their sum $\sum_{i=0}^{n} x_i$ is usually required (Xia, Liu, Wang, & Lai, 2000).

One problem with this encoding method is that many chromosomes decode into the same portfolio. This multiplies the GA's search space and makes GA less efficient in finding the optimal portfolio. Another problem with normalization is that similar chromosomes may decode into very different portfolios which makes it more difficult for GA to produce better chromosomes from good chromosomes.

3. Investment strategy portfolio problem and combination genetic algorithms

3.1. Investment strategy portfolio problem

The classical portfolio problem is a problem of distributing capital to a set of securities. By generalizing the set of securities to a set of investment strategies (or security-rule pairs), this study proposes an investment strategy portfolio problem, which becomes a problem of distributing capital to a set of investment strategies.

An investment strategy is a security-rule pair. A (trading) rule consists of a buy condition and a sell condition which are used to, respectively, determine the buying time and selling time for the target security. The buy/sell conditions can be based on various factors.

The classical portfolio problem can then be viewed a special case of the new investment strategy portfolio problem with buy-and-hold as the only trading rule. Other variants of the investment strategy portfolio problem include multiple securities with a single trading rule, a single security with multiple trading rules, and multiple securities with multiple trading rules.

3.2. Combination genetic algorithms

The real number model of portfolio problem in (1) can be approximated by the following integer model with a large m to achieve the desired precision.

$$\begin{aligned} \max \quad & \alpha \sum_i \frac{w_i}{m} \cdot r_i - (1-\alpha) \sum_{ij} \frac{w_i}{m} \cdot \frac{w_j}{m} \cdot \sigma_{ij} \\ \text{subject to} \quad & \sum_{i=0}^{n} w_i = m,\ 0 \leqslant w_i \leqslant m,\ w_i \in Z, \end{aligned} \tag{2}$$

where m is a large positive integer, w_i/m approximates the x_i in (1) and the constraints become $\sum_{i=0}^{n} w_i = m$, $0 \leqslant w_i \leqslant m$, & $w_i \in Z$. This turns the portfolio optimization problem into the combination optimization problem and reduces the search space to non-negative integers such that $\sum_{i=0}^{n} w_i = m$, whose size is C_m^{n+m}.

There are four common types of combinatorial problems: arrangement, permutation, combination, and combination with repetition. The sizes of their solution spaces are listed below.

Arrangement: There are n^r ways of ordering r of n distinct objects with repetitions.

Permutation: There are $P(n,r) = n!/(n-r)!$ ways of ordering r of n distinct objects without repetitions.

Combination: There are $C(n,r) = n!/r!(n-r)!$ ways of selecting r of n distinct objects without repetitions.

```
procedure BasicGA
  Initialization: Generate a random population of n chromosomes
  while (termination condition is not satisfied)
    Evaluation: Evaluate the fitness of each chromosome in the population
    loop (do genetic applications k times for each generation)
      Selection: Select 2 chromosomes according to their fitness
      Crossover: Cross over selected chromosomes to form new offspring with a crossover rate
      Mutation: Mutate each position in each new offspring with a mutation rate
    endloop
    Replacement: Replace chromosomes in parent population with new offspring
  endwhile
  Report the best solution (chromosome) found
endprocedure
```

Fig. 1. The basic genetic algorithm.

Combination with repetitions: There are $C(n+r-1,r)$ ways of selecting r of n distinct objects with repetitions.

Arrangement problems can be solved naturally by standard genetic algorithms. Permutation genetic algorithms have been developed to solve permutation optimization problems. However, few studies have been done on using genetic algorithms for combination optimization problems.

Standard GAs are not suitable for solving the combination optimization problem. We therefore propose a combination GA to solve the combination optimization problem. Its encoding and genetic operators are developed accordingly.

3.2.1. Combination encoding

The integer solutions for the constraint $\sum_{i=0}^{n} w_i = m$ have a one-to-one correspondence with the $n+m$-bit strings containing exactly m 1s. The correspondence can be established by treating the 0s as delimiters for w_i's and the consecutive 1s as unary numbers for w_i's. For example, the chromosome 1110110 represents that $w_0 = 3$, $w_1 = 2$ and $w_2 = 0$. We therefore use binary strings (of length $n+m$) with a fixed number (m) of 1s as encoding.

The implication of this combination encoding scheme is that each chromosome must have a certain number (m) of 1s initially, after crossover, and after mutation. Initial chromosome can be easily generated by randomly setting m of $n+m$ bits to 1s. The exchange mutation, which exchanges two randomly selected genes, for permutation encoding can also be used for combination encoding. The crossover operator for combination encoding requires special attention to conform to the m 1s constraint.

3.2.2. Combination crossover

Standard crossover operators for binary encoding have a high probability of violating the m 1s constraint. There are several ways to conform to the m 1s constraint. One way is to restrict the crossover points so that the crossovered sections have the same number of 1s. When the selected crossover points do not satisfy this condition, the crossover points are reselected until they do. This may require a large number of reselecting crossover points.

Another way to conform to the m 1s constraint is to repair the violating chromosomes after crossover. Assume that the violating chromosome has k 1s. If there are fewer than m 1s (i.e., $k < m$), then $m-k$ 0s are randomly selected to be changed to 1s. One the other hand, if there are more than m 1s (i.e., $k > m$), then $k-m$ 1s are randomly selected to be changed to 0s. Although the repair makes a chromosome feasible, it also makes the chromosome less similar to its parents.

Instead of the above two methods, we propose a shrinking crossover (SX). SX revises the two-point crossover to exchange the same number of 1s by moving the second crossover point leftward until there are equal number of 1s between the two crossover points in both parents. Fig. 2 gives the algorithm of the shrinking crossover.

```
procedure SX (C1, C2)
  select two crossover points s & t
  until (C1 & C2 have equal number of 1s between s & t)
    t = t-1
  enduntil
  for (i = s to t)
    Temp = C1 [i]
    C1 [i] = C2 [i]
    C2 [i] = Temp
  endfor
endprocedure
```

Fig. 2. Algorithm of the shrinking crossover (SX).

4. Experiments

4.1. Experiment 1: A single security with multiple trading rules

4.1.1. Security

The top priced stock from constituents of the Dow Jones Industrial Average, IBM, is used in this experiment as the only investment target. The data period is from July 2001 to December 2005.

4.1.2. Trading rules

The trading rules in our experiments are based on technical indicators. The following 10 technical indicators are selected: CCI, MOM, RSI, DMI, MACD, KD, WMS %R, BIAS, Qstick and PSY (Colby, 2002). Each technical indicator is used to form a buy signal and a sell signal as shown in Table 1. A trading rule consists of a buy signal from any technical indicator and a sell signal from any technical indicator. For example, a trading rule can have "CCI rises above ($\nearrow$) -100" as its buy signal and "RSI falls below ($\searrow$) 70" as its sell signal. As a result, the 10 buy signals and the 10 sell signals can form 100 different trading rules.

4.1.3. The CGA parameters

The investment strategy portfolios are generated and evaluated using the sliding time window method as shown in Fig. 3 (Kimoto, Asakawa, Yoda, & Takeoka, 1990). Portfolios are generated using $\ell = 9$ months' historical data and are tested for $s = 3$ month before new investment strategy portfolios are generated again. This process is repeated to move with time every 3 month for the whole $k = 12$ testing period. The returns are calculated after all transaction costs and taxes are deducted.

We use 1 stock and 100 trading rules to form a total of 100 security-rule pairs. However, only the best 10% performing pairs whose last training period's returns are ranked among the top 10 are used to construct the investment strategy portfolios. As a result, the number n of selected investment strategies is 10 with each $i = 1, \ldots, 10$ designating the ith ranked investment strategy and $i = 0$ designating the reserved cash. The total capital is divided into $m = 50$ equal portions.

Table 1
Trading signals of technical indicators

No.	Indicator	Buy signal	Sell signal
1	CCI	CCI $\nearrow -100$	CCI $\searrow 100$
2	MOM	MOM $\nearrow 0$	MOM $\searrow 0$
3	RSI	RSI line $\nearrow 30$	RSI line $\searrow 70$
4	DMI	+DI $\nearrow$ −DI	+DI $\searrow$ −DI
5	MACD	DIF $\nearrow$ MACD or 0	DIF $\searrow$ MACD or 0
6	KD	K $\nearrow$ D & D < 20	K $\searrow$ D & D > 20
7	WMS %R	WMS %R $\searrow$ 80	WMS %R $\nearrow$ 20
8	BIAS	BIAS $\nearrow -4.5\%$	BIAS $\searrow 5\%$
9	Qstick	Short Qstick $\nearrow$ Long Qstick or 0	Short Qstick $\searrow$ Long Qstick or 0
10	PSY	PSY $\nearrow 25\%$	PSY $\searrow 75\%$

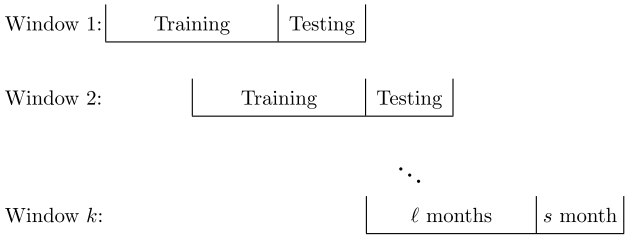

Fig. 3. Sliding time window simulation.

The weights of a portfolio are represented in a chromosome as a 60-bit binary string with 50 1s. The population size is 200. The selection method is roulette wheel. The crossover method is our SX crossover with a crossover rate of 100%. The mutation method is exchange mutation with a mutation rate of 0.5% per bit. Termination condition is simply fixed generations and the number of generations is set to 500.

The fitness of a chromosome is evaluated by its performance in the training period. We use the final wealth as the fitness function of our combination GA, which is defined as

$$f(w_0, w_1, \ldots, w_n) = \prod_{t=1}^{\ell}(1 + r_p(t)),$$

where $r_p(t) = \sum_{i=0}^{n} w_i r_i(t)$ is the return of the portfolio during month t in the training period.

4.1.4. Experimental results

To determine the average performance of combination GA on the investment strategy portfolio problem, the simulation is repeated five times with different initial random populations. The accumulated returns of combination GA on the single security strategy portfolio problem during the whole testing period range from −9.01% to −10.14% with an average of −9.50% as shown in Table 2. Table 3 compares the returns between CGA and uniform allocation on the single security strategy portfolio problem. The average accumulated returns of CGA during the whole testing period (−9.50%) is better than the accumulated returns of uniform allocation during the same testing period (−11.42%). In addition, paired t test between CGA and uniform allocation has a p value of 0.001958 which is much smaller than 0.01. As a result, the return of CGA is significantly better than that of uniform allocation at the 99% confidence level.

Table 2
Average portfolio returns of CGA on the single security problem

Testing period	Run1	Run2	Run3	Run4	Run5	Average
2003/01/01–03/31	−0.73	−1.62	−1.70	−1.72	−1.89	−1.53
2003/04/01–06/30	3.17	2.99	3.50	2.84	3.20	3.14
2003/07/01–09/30	−0.31	−1.40	−1.87	−0.97	−0.50	−1.01
2003/10/01–12/31	−1.02	−0.35	−0.78	−0.72	−0.24	−0.62
2004/01/01–03/31	−0.81	−1.15	−1.65	−1.16	−1.84	−1.32
2004/04/01–06/30	−4.93	−3.60	−3.49	−1.68	−1.97	−3.13
2004/07/01–09/30	−0.40	−0.24	−0.72	−1.31	−0.86	−0.71
2004/10/01–12/31	0.02	0.00	0.00	0.11	0.04	0.04
2005/01/01–03/31	−3.95	−4.25	−3.24	−4.75	−5.65	−4.37
2005/04/01–06/30	−1.08	−0.54	−1.46	−0.88	−2.53	−1.30
2005/07/01–09/30	0.60	1.01	1.01	0.49	0.97	0.81
2005/10/01–12/31	−0.31	−0.64	0.84	0.53	0.89	0.26
Accumulated	−9.54	−9.56	−9.34	−9.01	−10.14	−9.50

Table 3
Comparisons between CGA and uniform allocation on the single security problem

Testing period	CGA	Uniform
2003/01/01–03/31	−1.53	−1.62
2003/04/01–06/30	3.14	−1.16
2003/07/01–09/30	−1.01	0.72
2003/10/01–12/31	−0.62	−0.91
2003	−0.09	−2.95
2004/01/01–03/31	−1.32	−1.14
2004/04/01–06/30	−3.13	−2.54
2004/07/01–09/30	−0.71	−1.43
2004/10/01–12/31	0.04	3.26
2004	−5.05	−1.93
2005/01/01–03/31	−4.37	−3.38
2005/04/01–06/30	−1.3	−4.36
2005/07/01–09/30	0.81	0.66
2005/10/01–12/31	0.26	0.06
2005	−4.59	−6.93
Accumulated	−9.50	−11.42

4.2. Experiment 2: Multiple securities with multiple trading rules

4.2.1. Securities

The top 10 priced stocks from constituents of the Dow Jones Industrial Average are used in this experiment. They are IBM, BA, MMM, CAT, MO, AIG, PG, XOM, JNJ and UTX as listed in Table 4. The data period is from July 2001 to December 2005.

4.2.2. Trading rules

The trading rules in this experiment are the same as in experiment 1 as shown in Table 1.

4.2.3. The CGA parameters

We use 10 stocks and 100 trading rules to form a total of 1000 security-rule pairs. However, only the best 10% performing pairs whose last training period's returns are ranked among the top 10% are used to construct the investment strategy portfolios. As a result, the number n of selected investment strategies is 100 with each $i = 1, \ldots, 100$ designating the ith ranked investment strategy and $i = 0$ designating the reserved cash. The total capital is divided into $m = 50$ equal portions.

The weights of a portfolio are represented in a chromosome as a 150-bit binary string with 50 1s. As a result, the GA search space becomes

$$C_{50}^{150} = \frac{150!}{50!100!} \approx 2.01 \times 10^{40}.$$

All the remaining CGA parameters are set to the same as in Experiment 1.

4.2.4. Experimental results

To determine the average performance of combination GA on the investment strategy portfolio problem, the simulation is repeated 5 times with different initial random populations. The accumulated returns of combination GA on the multiple securities strategy portfolio problem during the whole testing period range from 15.65% to 17.22% with an average of 16.68% as shown in Table 5.

Table 6 compares the returns between CGA and uniform allocation on the multiple securities strategy portfolio problem. The average accumulated returns of CGA during the whole testing period (16.68%) is better than the accumulated returns of uniform allocation during the same testing period (12.06%). In addition, paired t test between CGA and uniform allocation has a p value of 0.00000749 which is much smaller than 0.01. As a result, the return of CGA is significantly better than that of uniform allocation at the 99% confidence level.

Table 4
Top 10 priced stocks of DJIA

No.	Symbol	Company name
1	IBM	International Business Machines Corp.
2	BA	Boeing Co.
3	MMM	3M Co.
4	CAT	Caterpillar Inc.
5	MO	Altria Group Inc.
6	AIG	American International Group, Inc.
7	PG	Procter & Gamble Co.
8	XOM	Exxon Mobil Corp.
9	JNJ	Johnson & Johnson DC
10	UTX	United Technologies Corp.

Table 5
Average portfolio returns of CGA on the multiple securities problem

Testing period	Run1	Run2	Run3	Run4	Run5	Average
2003/01/01–03/31	−0.03	−0.26	−0.72	0.24	−1.60	−0.47
2003/04/01–06/30	−1.72	−0.47	−2.87	−1.31	−0.86	−1.45
2003/07/01–09/30	2.31	5.56	2.78	3.5	3.49	3.53
2003/10/01–12/31	3.42	4.12	5.72	4.49	5.22	4.59
2004/01/01–03/31	−1.96	−1.81	−1.72	−2.07	−2.03	−1.92
2004/04/01–06/30	−1.46	−0.27	−2.8	−0.53	−1.32	−1.28
2004/07/01–09/30	−0.65	0.32	0.14	0.37	1.21	0.28
2004/10/01–12/31	5.7	1.61	4.21	3.25	5.03	3.96
2005/01/01–03/31	0.69	1.21	0.33	−1.16	−0.56	0.1
2005/04/01–06/30	4.17	2.13	2.99	2.02	1.46	2.55
2005/07/01–09/30	3.08	2.38	5.01	5.92	4.19	4.12
2005/10/01–12/31	2.7	1.51	2.02	0.73	2.14	1.82
Accumulated	17.11	16.99	15.65	16.18	17.22	16.68

Table 6
Comparisons between CGA and uniform allocation on the multiple securities problem

Testing period	CGA	Uniform
2003/01/01–03/31	−0.47	−0.58
2003/04/01–06/30	−1.45	−1.4
2003/07/01–09/30	3.53	3.27
2003/10/01–12/31	4.59	5.55
2003	6.21	6.85
2004/01/01–03/31	−1.92	−2.45
2004/04/01–06/30	−1.28	−1.35
2004/07/01–09/30	0.28	−0.45
2004/10/01–12/31	3.96	3.15
2004	0.94	−1.18
2005/01/01–03/31	0.10	−0.02
2005/04/01–06/30	2.55	1.9
2005/07/01–09/30	4.12	2.8
2005/10/01–12/31	1.82	1.34
2005	8.83	6.14
Accumulated	16.68	12.06

5. Conclusion

This study proposes a new investment strategy portfolio problem generalized from the classical portfolio problem. In addition, a new combination genetic algorithm is proposed for solving this new investment strategy portfolio problem. Experimental results have demonstrated the feasibility of the investment strategy portfolio idea and the effectiveness of our combination genetic algorithm on the investment strategy portfolio problem.

Future work includes the incorporation of short trading rules in addition to current long trading rules. Generating variable weights instead of just constant weights is another interesting direction.

Acknowledgement

This work was supported by a Grant (NSC96-2416-H-008-017) from the National Science Council of Taiwan.

References

Bäck, T., Fogel, D. B., & Michalewicz, T. (Eds.). (2000a). *Evolutionary computation 1: Basic algorithms and operators*. IOP.
Bäck, T., Fogel, D. B., & Michalewicz, T. (Eds.). (2000b). *Evolutionary computation 2: Advanced algorithms and operators*. IOP.
Bauer, R. J. Jr., (1994). *Genetic algorithms and investment strategies*. Wiley.
Beasley, D. (2000). Possible applications of evolutionary computation. In T. Bäck, D. B. Fogel, & T. Michalewicz (Eds.), *Evolutionary computation 2: Advanced algorithms and operators* (pp. 4–19). IOP.
Colby, R. W. (2002). *The encyclopedia of technical market indicators* (2nd ed.). McGraw-Hill.
Colin, A. M. (1994). Genetic algorithms for financial modeling. In G. J. Deboeck (Ed.), *Trading on the edge: Neural, genetic and fuzzy systems for chaotic financial markets* (pp. 148–173). Wiley.
Darwin, C. (Ed.). (1859). *On the origin of species by means of natural selection, or the preservation of favoured races in the struggle for life*. University of Virginia Library.
De Jong, K. A. (Ed.). (2002). *Evolutionary computation: A unifiede approach*. MIT Press.
Deboeck, G. J. (1994). Using GAs to optimize a trading system. In G. J. Deboeck (Ed.), *Trading on the edge: Neural, genetic and fuzzy systems for chaotic financial markets* (pp. 174–188). Wiley.
Goldberg, D. E. (1989). *Genetic algorithms in search, optimization and machine learning*. Addison-Wesley.
Gondzio, J., & Grothey, A. (2007). Solving non-linear portfolio optimization problems with the primal-dual interior point method. *European Journal of Operational Research, 181*(3), 1019–1029.
Holland, J. H. (1975). *Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control, and artificial intelligence*. University of Michigan Press.
Ince, H., & Trafalis, T. B. (2006). Kernel methods for short-term portfolio management. *Expert Systems with Applications, 30*, 535–542.
Kim, G., & Markowitz, H. M. (1989). Investment rules, margin, and market volatility. *Journal of Portfolio Management, 16*(1), 45–52.
Kimoto, T., Asakawa, K., Yoda, M., & Takeoka, M. (1990). Stock market prediction system with modular neural networks. In *Proceedings of the 1990 international joint conference on neural networks* (pp. 1–6).
Markowitz, H. M., & Arnott, R. D. (1952). Portfolio selection. *Journal of Finance, 7*(7), 77–91.
Mitchell, M. (1996). *An introduction to genetic algorithms*. MIT Press.
Oh, K. J., Kim, T. Y., & Min, S. (2005). Using genetic algorithm to support portfolio optimization for index fund management. *Expert Systems with Applications, 28*(2), 371–379.
Oh, K. J., Kim, T. Y., Min, S.-H., & Lee, H. Y. (2006). Portfolio algorithm based on portfolio beta using genetic algorithm. *Expert Systems with Applications, 30*(3), 527–534.
Srinivas, M., & Patnaik, L. (1994). Genetic algorithms: A survey. *Computer, 27*(6), 17–26.
Wu, M.-C., & Chang, W.-J. (2007). A short-term capacity trading method for semiconductor fabs with partnership. *Expert Systems with Applications, 33*(2), 476–483.
Xia, Y., Liu, B., Wang, S., & Lai, K. K. (2000). A model for portfolio selection with order of expected returns. *Computers and Operations Research*, 409–422.